

# TUGAS WEB SERVICE

Dokumentasi dan Source Code

Proyek NestJS — API Destinasi

Dokumen ini dibuat dari arsip **tugas-web-service.zip**. Folder dependensi, repository Git, dan hasil build tidak disertakan agar PDF tetap ringkas dan mudah dibaca.

# 1. Gambaran Proyek

Proyek merupakan Web Service berbasis NestJS dan TypeScript dengan resource **destinasi**. API menyediakan operasi untuk mengambil daftar destinasi, membuat data baru, mengambil berdasarkan ID, memperbarui, dan menghapus data. Dokumentasi API dibuat menggunakan Swagger/OpenAPI.

## Struktur utama

| Folder/File           | Keterangan                                                   |
|-----------------------|--------------------------------------------------------------|
| src/main.ts           | Entry point aplikasi dan konfigurasi Swagger/ValidationPipe. |
| src/app.module.ts     | Module utama aplikasi.                                       |
| src/app.controller.ts | Controller endpoint root (/).                                |
| src/app.service.ts    | Service untuk endpoint aplikasi.                             |
| src/destinasi/        | Controller dan DTO untuk resource destinasi.                 |
| test/                 | Pengujian end-to-end.                                        |
| openapi.json          | Spesifikasi OpenAPI yang dihasilkan aplikasi.                |
| package.json          | Script npm dan daftar dependency.                            |

# 2. Ringkasan Endpoint API

Judul API: **API Destinasi** — versi 1.0. Deskripsi: Dokumentasi API Web Service.

| Method | Endpoint        | Fungsi                                    |
|--------|-----------------|-------------------------------------------|
| GET    | /               | AppController_getHello                    |
| GET    | /destinasi      | Mendapatkan semua destinasi               |
| POST   | /destinasi      | Membuat destinasi baru                    |
| GET    | /destinasi/{id} | Mendapatkan data destinasi berdasarkan ID |
| PATCH  | /destinasi/{id} | Memperbarui data destinasi berdasarkan ID |
| DELETE | /destinasi/{id} | Menghapus data destinasi berdasarkan ID   |

## Schema DTO

| Schema             | Field | Tipe   | Wajib | Contoh      |
|--------------------|-------|--------|-------|-------------|
| CreateDestinasiDto | nama  | string | Ya    | Pantai Kuta |
| UpdateDestinasiDto | nama  | string | Tidak | Pantai Kuta |

### 3. Source Code & Konfigurasi

Bagian berikut memuat file sumber dan konfigurasi penting dari proyek. Kode ditampilkan dalam format monospace agar mudah dibaca saat dicetak.

#### package.json

```
{
  "name": "tugas-web-service",
  "version": "0.0.1",
  "description": "",
  "author": "",
  "private": true,
  "license": "UNLICENSED",
  "type": "module",
  "scripts": {
    "build": "nest build",
    "deploy": "nest deploy",
    "format": "prettier --write \"src/**/*.ts\" \"test/**/*.ts\"",
    "start": "nest start",
    "start:dev": "nest start --watch",
    "start:debug": "nest start --debug --watch",
    "start:prod": "node dist/main",
    "lint": "oxlint --type-aware src/ test/",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:cov": "vitest run --coverage",
    "test:debug": "vitest --inspect-brk --no-file-parallelism",
    "test:e2e": "vitest run --config ./vitest.config.e2e.ts"
  },
  "dependencies": {
    "@nestjs/common": "^12.0.1",
    "@nestjs/core": "^12.0.1",
    "@nestjs/observe": "^0.3.0",
    "@nestjs/platform-express": "^12.0.1",
    "@nestjs/swagger": "^12.0.2",
    "class-transformer": "^0.5.1",
    "class-validator": "^0.15.1",
    "reflect-metadata": "^0.2.2",
    "rxjs": "^7.8.1",
    "swagger-ui-express": "^5.0.1"
  },
  "devDependencies": {
    "@nestjs/cli": "^12.0.0",
    "@nestjs/mau": "^0.2.6",
    "@nestjs/schematics": "^12.0.0",
    "@nestjs/testing": "^12.0.1",
    "@types/express": "^5.0.0",
    "@types/node": "^24.0.0",
    "@types/supertest": "^7.0.0",
    "@vitest/coverage-v8": "^4.1.2",
    "oxlint": "^1.58.0",
    "oxlint-tsgolint": "^7.0.2001",
    "prettier": "^3.4.2",
    "source-map-support": "^0.5.21",
    "supertest": "^7.0.0",
    "typescript": "^6.0.2",
    "vite-tsconfig-paths": "^5.1.4",
    "vitest": "^4.1.2"
  }
}
```

#### nest-cli.json

```
{
  "$schema": "https://json.schemastore.org/nest-cli",
  "collection": "@nestjs/schematics",
  "sourceRoot": "src",
  "compilerOptions": {
    "deleteOutDir": true
  }
}
```

#### tsconfig.json

```
{
  "compilerOptions": {
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "resolvePackageJsonExports": true,

```

```

    "esModuleInterop": true,
    "isolatedModules": true,
    "declaration": true,
    "removeComments": true,
    "emitDecoratorMetadata": true,
    "experimentalDecorators": true,
    "allowSyntheticDefaultImports": true,
    "target": "ES2023",
    "sourceMap": true,
    "outDir": "./dist",
    "rootDir": ".",
    "incremental": true,
    "skipLibCheck": true,
    "strict": true,
    "strictPropertyInitialization": false,
    "types": ["vitest/globals", "node"]
  }
}

```

## tsconfig.build.json

```

{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "rootDir": "./src"
  },
  "include": ["src"],
  "exclude": ["node_modules", "test", "dist", "**/*spec.ts"]
}

```

## .prettierrc

```

{
  "singleQuote": true,
  "trailingComma": "all"
}

```

## .oxlintrc.json

```

{
  "$schema": "./node_modules/oxlint/configuration_schema.json",
  "rules": {
    "typescript/no-explicit-any": "off",
    "typescript/no-floating-promises": "error"
  },
  "env": {
    "node": true
  }
}

```

## src/main.ts

```

import { NestFactory } from '@nestjs/core';
import { AppModule, ObserveInstrument } from './app.module.js';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { ValidationPipe } from '@nestjs/common';
import { writeFileSync } from 'node:fs';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    instrument: ObserveInstrument,
  });
  app.useGlobalPipes(new ValidationPipe());

  const config = new DocumentBuilder()
    .setTitle('API Destinasi')
    .setDescription('Dokumentasi API Web Service')
    .setVersion('1.0')
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api/docs', app, document);
  writeFileSync('./openapi.json', JSON.stringify(document, null, 2));

  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();

```

## src/app.controller.ts

```
import { Controller, Get } from '@nestjs/common';
import { AppService } from '../app.service.js';

@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

### src/app.module.ts

```
import { Module } from '@nestjs/common';
import { createObservModule } from '@nestjs/observe';
import { AppController } from '../app.controller.js';
import { AppService } from '../app.service.js';
import { DestinasiController } from '../destinasi/destinasi.controller.js';

export const { ObservModule, ObservInstrument } = createObservModule();

@Module({
  imports: [
    // Distributed tracing, auto-correlated logs, request/job metrics, error
    // telemetry, alarms, and more – out of the box. Sign up at https://observe.nestjs.com
    ObservModule.forRoot({
      appKey: 'YOUR_APP_KEY',
      appSecret: 'YOUR_APP_SECRET',
      serviceId: 'tugas-web-service',
    }),
  ],
  controllers: [AppController, DestinasiController],
  providers: [AppService],
})
export class AppModule {}
```

### src/app.service.ts

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}
```

### src/app.controller.spec.ts

```
import { Test, TestingModule } from '@nestjs/testing';
import { AppController } from '../app.controller.js';
import { AppService } from '../app.service.js';

describe('AppController', () => {
  let appController: AppController;

  beforeEach(async () => {
    const app: TestingModule = await Test.createTestingModule({
      controllers: [AppController],
      providers: [AppService],
    }).compile();

    appController = app.get<AppController>(AppController);
  });

  describe('root', () => {
    it('should return "Hello World!"', () => {
      expect(appController.getHello()).toBe('Hello World!');
    });
  });
});
```

### src/destinasi/destinasi.controller.ts

```
import { Body, Controller, Delete, Get, Param, Patch, Post } from '@nestjs/common';
import { ApiBody, ApiOperation, ApiResponse, ApiTags } from '@nestjs/swagger';
import { CreateDestinasiDto } from '../dto/create-destinasi.dto.js';
import { UpdateDestinasiDto } from '../dto/update-destinasi.dto.js';
```

```

@ApiTags('destinasi')
@Controller('destinasi')
export class DestinasiController {
  @Get()
  @ApiOperation({ summary: 'Mendapatkan semua destinasi' })
  @ApiResponse({ status: 200, description: 'Daftar destinasi.' })
  findAll() {
    return [{ id: 1, nama: 'Pantai Kuta' }];
  }

  @Post()
  @ApiOperation({ summary: 'Membuat destinasi baru' })
  @ApiBody({ type: CreateDestinasiDto })
  @ApiResponse({ status: 201, description: 'Destinasi berhasil dibuat.' })
  create(@Body() createDestinasiDto: CreateDestinasiDto) {
    return { id: 1, ...createDestinasiDto };
  }

  @Get(':id')
  @ApiOperation({ summary: 'Mendapatkan data destinasi berdasarkan ID' })
  @ApiResponse({ status: 200, description: 'Data destinasi berhasil ditemukan.' })
  findOne(@Param('id') id: string) {
    return { id, nama: 'Pantai Kuta' };
  }

  @Patch(':id')
  @ApiOperation({ summary: 'Memperbarui data destinasi berdasarkan ID' })
  @ApiBody({ type: UpdateDestinasiDto })
  @ApiResponse({ status: 200, description: 'Data destinasi berhasil diperbarui.' })
  update(
    @Param('id') id: string,
    @Body() updateDestinasiDto: UpdateDestinasiDto,
  ) {
    return { id, nama: 'Pantai Kuta', ...updateDestinasiDto };
  }

  @Delete(':id')
  @ApiOperation({ summary: 'Menghapus data destinasi berdasarkan ID' })
  @ApiResponse({ status: 200, description: 'Data destinasi berhasil dihapus.' })
  remove(@Param('id') id: string) {
    return { message: `Berhasil menghapus data dummy destinasi ID ${id}` };
  }
}

```

### src/destinasi/dto/create-destinasi.dto.ts

```

import { ApiProperty } from '@nestjs/swagger';
import { IsString, IsNotEmpty } from 'class-validator';

export class CreateDestinasiDto {
  @ApiProperty({ example: 'Pantai Kuta', description: 'Nama destinasi wisata' })
  @IsString()
  @IsNotEmpty()
  nama: string;
}

```

### src/destinasi/dto/update-destinasi.dto.ts

```

import { PartialType } from '@nestjs/swagger';
import { CreateDestinasiDto } from '../create-destinasi.dto.js';

export class UpdateDestinasiDto extends PartialType(CreateDestinasiDto) {}

```

### src/destinasi/main.ts

```

import { NestFactory } from '@nestjs/core';
import { AppModule } from '../app.module.js';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import * as fs from 'fs';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // Konfigurasi Swagger
  const config = new DocumentBuilder()
    .setTitle('API Destinasi')
    .setDescription('Dokumentasi API Web Service')
    .setVersion('1.0')
    .build();
  const document = SwaggerModule.createDocument(app, config);

  // Setup endpoint Swagger di http://localhost:3000/api/docs
}

```

```

SwaggerModule.setup('api/docs', app, document);

// Ekspor dokumen OpenAPI ke file openapi.json
fs.writeFileSync('./openapi.json', JSON.stringify(document, null, 2));

await app.listen(3000);
}
bootstrap();

```

## test/app.e2e-spec.ts

```

import { Test, TestingModule } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import request from 'supertest';
import { App } from 'supertest/types';
import { AppModule } from '../src/app.module.js';

describe('AppController (e2e)', () => {
  let app: INestApplication<App>;

  beforeEach(async () => {
    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleFixture.createNestApplication();
    await app.init();
  });

  it('/ (GET)', () => {
    return request(app.getHttpServer())
      .get('/')
      .expect(200)
      .expect('Hello World!');
  });

  afterEach(async () => {
    await app.close();
  });
});

```

## vitest.config.ts

```

import { defineConfig } from 'vitest/config';
import tsconfigPaths from 'vite-tsconfig-paths';

export default defineConfig({
  // Resolves the path aliases declared in tsconfig.json, including the ones
  // added by `nest g library`.
  plugins: [tsconfigPaths()],
  test: {
    globals: true,
    root: './',
    include: ['**/*.spec.ts'],
  },
});

```

## vitest.config.e2e.ts

```

import { defineConfig } from 'vitest/config';
import tsconfigPaths from 'vite-tsconfig-paths';

export default defineConfig({
  plugins: [tsconfigPaths()],
  test: {
    globals: true,
    root: './',
    include: ['**/*.e2e-spec.ts'],
  },
});

```

## README.md

```

<p align="center">
  <a href="http://nestjs.com/" target="blank"></a>
</p>
[circleci-image]: https://img.shields.io/circleci/build/github/nestjs/nest/master?token=abc123def456
[circleci-url]: https://circleci.com/gh/nestjs/nest
<p align="center">A progressive <a href="http://nodejs.org" target="_blank">Node.js</a> framework for building efficient and scalable

```

```

<p align="center">■
<a href="https://www.npmjs.com/~nestjsscore" target="_blank">
<a href="https://www.npmjs.com/~nestjscore" target="_blank">
<a href="https://www.npmjs.com/~nestjscore" target="_blank">
<a href="https://circleci.com/gh/nestjs/nest" target="_blank">
<a href="https://discord.gg/G7Qnnhy" target="_blank">
<a href="https://opencollective.com/nest#backer" target="_blank">
<a href="https://opencollective.com/nest#sponsor" target="_blank">
<a href="https://opencollective.com/nest#sponsor" target="_blank">
<a href="https://twitter.com/nestframework" target="_blank">
</p>■
<!--[!Backers on Open Collective](https://opencollective.com/nest/backers/badge.svg)][https://opencollective.com/nest#backer]■
[!Sponsors on Open Collective](https://opencollective.com/nest/sponsors/badge.svg)][https://opencollective.com/nest#sponsor]-->■
■
## Description■
■
[Nest](https://github.com/nestjs/nest) framework TypeScript starter repository.■
■
### Project setup■
■
```bash■
$ npm install■
```■
■
## Compile and run the project■
■
```bash■
# development■
$ npm run start■
■
# watch mode■
$ npm run start:dev■
■
# production mode■
$ npm run start:prod■
```■
■
## Run tests■
■
```bash■
# unit tests■
$ npm run test■
■
# e2e tests■
$ npm run test:e2e■
■
# test coverage■
$ npm run test:cov■
```■
■
## Deployment■
■
When you're ready to deploy your NestJS application to production, there are some key steps you can take to ensure it runs as efficiently as possible.
■
If you are looking for a cloud-based platform to deploy your NestJS application, check out [Mau](https://mau.nestjs.com), our official deployment tool.
■
```bash■
$ npm install -g @nestjs/mau■
$ mau deploy■
```■
■
With Mau, you can deploy your application in just a few clicks, allowing you to focus on building features rather than managing infrastructure.
■
## Observability■
■
In production applications, observability is essential for understanding how your system behaves, detecting issues early, and maintaining high performance.
■
[NestJS Observe](https://observe.nestjs.com) automatically instruments your NestJS application, giving you deep visibility into your system's performance and health.
■
- **Distributed tracing:** Follow requests across services and understand how they flow through your system.■
- **Waterfall analysis:** Visualize request execution and identify slow operations, bottlenecks, and unexpected delays.■
- **Performance analysis:** Analyze application performance in real time and quickly pinpoint areas that need optimization.■
- **Metrics:** Track key application and infrastructure metrics to understand system health and performance trends.■
- **Logging:** Centralize and correlate logs with traces and other telemetry to make debugging easier.■
- **Error tracking:** Detect errors quickly and investigate their root causes with the surrounding context.■
- **SLA monitoring:** Track service-level objectives and identify when your application is approaching or exceeding defined thresholds.■
- **Alarms and alerts:** Set up alerts for critical errors, performance degradation, SLA violations, and other anomalies so your team can respond quickly.■
This project is already instrumented. Create a free account at [observe.nestjs.com](https://observe.nestjs.com), add an application, and start monitoring.
■
The free plan needs no payment details and covers 300,000 events a month. You can also browse the [live demo](https://www.observe-demo.nestjs.com).
■
## Resources■
■
Check out a few resources that may come in handy when working with NestJS:■
- Visit the [NestJS Documentation](https://docs.nestjs.com) to learn more about the framework.■

```

- For questions and support, please visit our [Discord channel](https://discord.gg/G7Qnnhy).
- To dive deeper and get more hands-on experience, check out our official video [courses](https://courses.nestjs.com/).
- Deploy your application to AWS with the help of [NestJS Mau](https://mau.nestjs.com) in just a few clicks.
- Auto-instrument your application with [NestJS Observe](https://observe.nestjs.com). Distributed tracing, metrics, and logging made
- Visualize your application graph and interact with the NestJS application in real-time using [NestJS Devtools](https://devtools.nestjs.com).
- Need help with your project (part-time to full-time)? Check out our official [enterprise support](https://enterprise.nestjs.com).
- To stay in the loop and get updates, follow us on [X](https://x.com/nestframework) and [LinkedIn](https://linkedin.com/company/nestjs).
- Looking for a job, or have a job to offer? Check out our official [Jobs board](https://jobs.nestjs.com).

■

## Support ■

■

Nest is an MIT-licensed open source project. It can grow thanks to the sponsors and support by the amazing backers. If you'd like to

■

## Stay in touch ■

■

- Author - [Kamil Myśliwiec](https://twitter.com/kammysliwiec) ■
- Website - [https://nestjs.com](https://nestjs.com) ■
- Twitter - [@nestjsframework](https://twitter.com/nestframework) ■

■

## License ■

■

Nest is [MIT licensed](https://github.com/nestjs/nest/blob/master/LICENSE). ■

## openapi.json

```
{
  "openapi": "3.0.0",
  "paths": {
    "/": {
      "get": {
        "operationId": "AppController_getHello",
        "parameters": [],
        "responses": {
          "200": {
            "description": ""
          }
        },
        "tags": [
          "App"
        ]
      }
    },
    "/destinasi": {
      "get": {
        "operationId": "DestinasiController_findAll",
        "parameters": [],
        "responses": {
          "200": {
            "description": "Daftar destinasi."
          }
        },
        "summary": "Mendapatkan semua destinasi",
        "tags": [
          "destinasi"
        ]
      },
      "post": {
        "operationId": "DestinasiController_create",
        "parameters": [],
        "requestBody": {
          "required": true,
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/CreateDestinasiDto"
              }
            }
          }
        },
        "responses": {
          "201": {
            "description": "Destinasi berhasil dibuat."
          }
        },
        "summary": "Membuat destinasi baru",
        "tags": [
          "destinasi"
        ]
      }
    },
    "/destinasi/{id}": {
      "get": {
        "operationId": "DestinasiController_findOne",
        "parameters": [
          {

```

```

        "name": "id",
        "required": true,
        "in": "path",
        "schema": {
            "type": "string"
        }
    },
    ],
    "responses": {
        "200": {
            "description": "Data destinasi berhasil ditemukan."
        }
    },
    "summary": "Mendapatkan data destinasi berdasarkan ID",
    "tags": [
        "destinasi"
    ]
},
"patch": {
    "operationId": "DestinasiController_update",
    "parameters": [
        {
            "name": "id",
            "required": true,
            "in": "path",
            "schema": {
                "type": "string"
            }
        }
    ],
    "requestBody": {
        "required": true,
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/UpdateDestinasiDto"
                }
            }
        }
    },
    "responses": {
        "200": {
            "description": "Data destinasi berhasil diperbarui."
        }
    },
    "summary": "Memperbarui data destinasi berdasarkan ID",
    "tags": [
        "destinasi"
    ]
},
"delete": {
    "operationId": "DestinasiController_remove",
    "parameters": [
        {
            "name": "id",
            "required": true,
            "in": "path",
            "schema": {
                "type": "string"
            }
        }
    ],
    "responses": {
        "200": {
            "description": "Data destinasi berhasil dihapus."
        }
    },
    "summary": "Menghapus data destinasi berdasarkan ID",
    "tags": [
        "destinasi"
    ]
}
}
},
"info": {
    "title": "API Destinasi",
    "description": "Dokumentasi API Web Service",
    "version": "1.0",
    "contact": {}
},
"tags": [],
"servers": [],
"components": {
    "schemas": {
        "CreateDestinasiDto": {
            "type": "object",
            "properties": {

```

```
{
    "nama": {
        "type": "string",
        "example": "Pantai Kuta",
        "description": "Nama destinasi wisata"
    },
    "required": [
        "nama"
    ],
    "UpdateDestinasiDto": {
        "type": "object",
        "properties": {
            "nama": {
                "type": "string",
                "example": "Pantai Kuta",
                "description": "Nama destinasi wisata"
            }
        }
    }
}
```